Analysis and Reproduction of CenterPoint for 3D Object Detection and Tracking

Erwan Ouabdesselam

Abstract

3D object detection from LiDAR point clouds is a fundamental component of autonomous driving perception systems. CenterPoint proposes a center-based formulation that models objects as keypoints and regresses their full 3D bounding boxes from a bird's-eye-view (BEV) representation. This report analyzes the theoretical principles of the method, implements key components of the pipeline, evaluates the model on the nuScenes dataset, and provides a critical assessment of its strengths and limitations. All implementation details, along with the code and visualizations used in this report, are available in the accompanying GitHub repository: https://github.com/ErwanR123/Analysis_of_CenterPoint.

1 Introduction

Autonomous driving systems rely on accurate 3D perception from LiDAR sensors, which provide sparse, irregular point clouds of the surrounding environment. Three challenges make this task significantly harder than 2D image detection: (i) most of the 3D space contains no measurements; (ii) objects appear with arbitrary orientations; and (iii) object classes span highly heterogeneous sizes, from bicycles to buses.

Many existing 3D detectors use *anchors*, predefined bounding-box candidates at fixed orientations, to hypothesize object locations. While effective in 2D where boxes align with image axes, this strategy is poorly suited to 3D: covering all plausible orientations requires a large number of anchors, increasing computational cost, complicating positive/negative sample assignment during training, and inflating the number of potential false positives.

CenterPoint [5] addresses these issues by reformulating 3D detection as *keypoint estimation*. Instead of predicting oriented boxes directly, the model first detects object *centers* in a bird's-eye-view heatmap and regresses their geometric attributes from those locations. Since a point carries no intrinsic orientation, this representation eliminates the need to enumerate orientation hypotheses and substantially reduces the detector's search space.

The main contributions of CenterPoint are: (i) a center-based detection head compatible with any standard 3D backbone; (ii) an efficient two-stage refinement module using surface-point features; and (iii) a unified detection-and-tracking framework via velocity estimation.

2 Analysis of the CenterPoint Pipeline

Figure 1 illustrates the overall architecture. The pipeline converts a raw LiDAR point cloud into a BEV feature map via voxelization and a sparse 3D backbone, then applies a center-based detection head followed by an optional second-stage refinement.

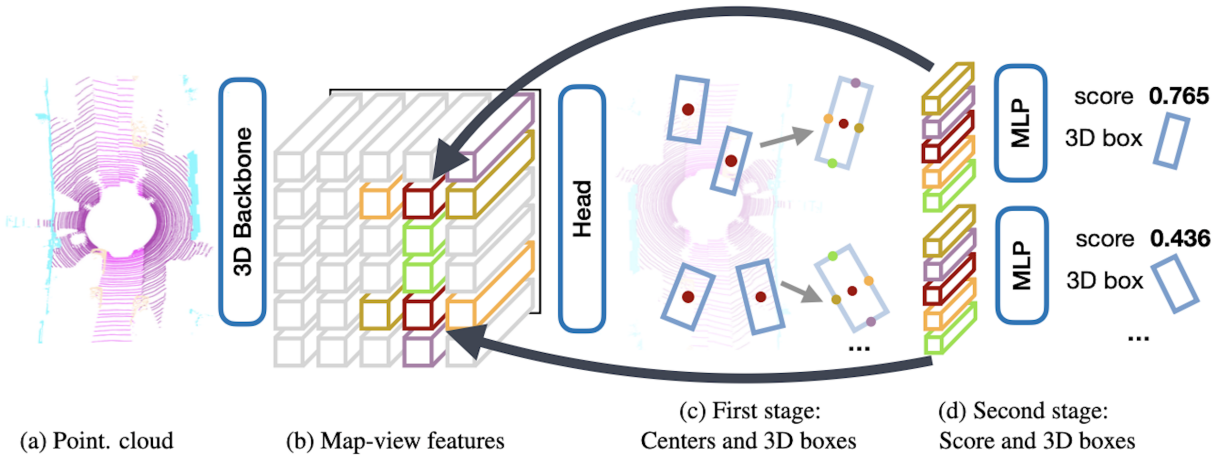

Figure 1: CenterPoint architecture. Source : Yin et al., "Center-based 3D Object Detection and Tracking", CVPR 2021.

2.1 Voxelization and Point Cloud Encoding

LiDAR point clouds are irregular and unordered sets of points, whereas convolutional neural networks require structured inputs. CenterPoint addresses this mismatch by discretizing the 3D space into volumetric cells called voxels. On nuScenes, 10 consecutive sweeps are aggregated to densify measurements, adding a timestamp offset Δt as a fifth point attribute, so that each point is fully described by $(x, y, z, r, \Delta t)$.

Only occupied voxels are stored. In our experiments, only $\approx 0.114\%$ of the theoretical voxel grid is occupied, motivating the use of sparse convolutions.

CenterPoint evaluates two voxelization strategies. **VoxelNet-style approach** discretizes the full 3D space into cubic voxels, preserving the complete vertical geometry of the scene. The **PointPillars-style approach** collapses the vertical dimension into tall vertical columns (pillars), producing a 2D pseudo-image that is computationally cheaper but loses height information. In this work we adopt the VoxelNet formulation, which preserves fine 3D structure at the cost of higher memory usage, a trade-off justified by the relatively small voxel size ($0.075\,\text{m} \times 0.075\,\text{m} \times 0.2\,\text{m}$) used on nuScenes.

The voxel size controls the fundamental trade-off between spatial resolution and computation. Figure 2 illustrates this effect: the CenterPoint configuration uses $0.075\,\text{m} \times 0.075\,\text{m} \times 0.2\,\text{m}$ voxels on nuScenes (grid size 1440×1440), preserving fine geometric detail. Coarser voxels at 20 cm or 50 cm render small objects (pedestrians, cyclists) as unresolvable blobs, directly degrading detection accuracy for those classes.

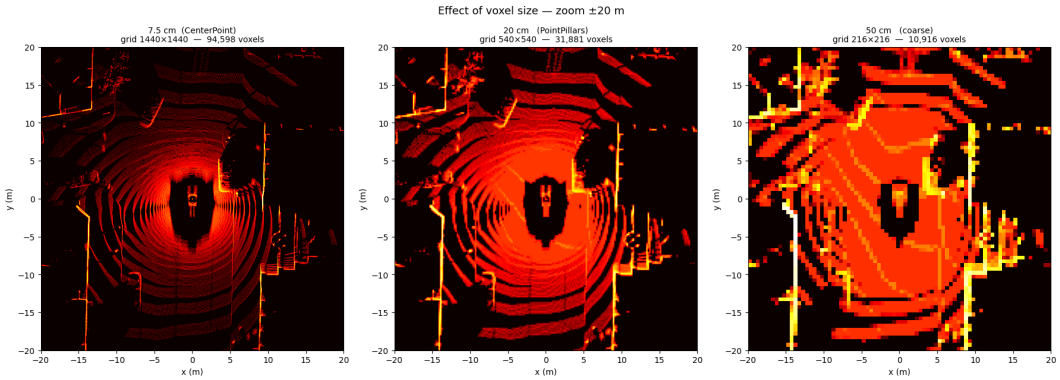

Figure 2: Effect of voxel resolution on BEV scene representation (7.5,cm, 20,cm, 50,cm from left to right). Geometric detail degrades rapidly with larger voxels, making small objects such as pedestrians and cyclists increasingly difficult to resolve

After voxelization, each occupied voxel contains a variable number of points (between 1 and `max_points=10`), all padded to the same length with zeros. The backbone requires exactly one fixed-size feature vector per voxel, which is the role of the **Voxel Feature Extractor (VFE)**.

CenterPoint uses the simplest possible aggregation: the *MeanVFE*, which averages the features of the real points in each voxel.

Training augmentation. Standard augmentations (random flipping, scaling, rotation) are applied to the point cloud before voxelization. To address nuScenes' long-tail class distribution, CenterPoint adopts ground-truth sampling [4]: annotated objects from other frames are pasted into the current scene, increasing the occurrence of rare classes during training. This directly explains the zero-AP results observed on trailer, construction vehicle, and barrier in our mini-val evaluation (Section 3.3), where these classes are entirely absent from the 2-scene split.

2.2 Sparse 3D Backbone and BEV Feature Map

The sparse voxel grid is processed by a sparse 3D convolutional backbone, consistent with the VoxelNet formulation adopted in Section 2.1. Sparse convolutions operate only on occupied voxels, making computation proportional to scene density rather than grid volume. The backbone progressively reduces spatial resolution through strided convolutions across several blocks, then collapses the vertical dimension to produce a dense 2D BEV feature map $M \in \mathbb{R}^{W \times L \times F}$, where W and L are the spatial dimensions directly determined by the voxel resolution and the backbone stride, and F the number of feature channels.

A 2D convolutional neck then upsamples and concatenates feature maps from different backbone strides, producing a richer multi-scale BEV representation that is passed to the detection head.

2.3 Center-Based Detection Head

As discussed in the introduction, anchor-based strategies are poorly suited to 3D detection. CenterPoint instead reformulates detection as a keypoint estimation problem following CenterNet [6]:

a point, unlike a box, carries no intrinsic orientation, allowing the backbone to learn rotational invariance more naturally.

Center heatmap. For each of K object classes, the head predicts a heatmap $\hat{Y} \in [0,1]^{W \times L \times K}$, where each spatial location encodes the probability that an object center of that class lies at that position in the BEV map. During training, a 2D Gaussian peak is rendered at the projected ground-truth center of each annotated object. The Gaussian radius is set to:

$$\sigma = \max\big(f(wl),\ \tau\big) \tag{1}$$

where $f(wl)$ is the CornerNet [2] radius function that scales the peak with the object's BEV footprint, and $\tau = 2$ is a minimum radius that ensures even small objects receive a sufficiently dense supervision signal. This adaptive radius is particularly important in map-view, where object centers are much sparser than in image-based detection: without it, the vast majority of locations would be treated as background, making the learning signal extremely sparse.

The heatmap is trained with a Focal Loss [3]:

$$\mathcal{L}_{\text{heatmap}} = -\frac{1}{N} \sum_{xyc} \begin{cases} (1-\hat{Y}_{xyc})^{\beta} \log(\hat{Y}_{xyc}) & \text{if } Y_{xyc} = 1 \\ (1-Y_{xyc})^{\alpha} \hat{Y}_{xyc}^{\beta} \log(1-\hat{Y}_{xyc}) & \text{otherwise} \end{cases} \tag{2}$$

The focal loss is essential here: since only a tiny fraction of BEV locations correspond to object centers, a standard cross-entropy loss would be dominated by background pixels, preventing the network from learning meaningful foreground features.

At inference, object centers correspond to local maxima in $\hat{Y}$, defined as locations whose heatmap value exceeds all 8 spatial neighbors. The confidence of each detection is proportional to the heatmap value at its peak.

Regression heads. For each detected center, dedicated regression heads predict the remaining parameters of the 3D bounding box. The predicted quantities are:

- **Sub-voxel offset** $o \in \mathbb{R}^2$: compensates for the quantization error introduced by voxelization and backbone striding, improving center localization precision;
- **Height above ground** $h_g \in \mathbb{R}$: recovers the elevation information lost when collapsing the vertical dimension into the BEV map;
- **3D box size** $(l, w, h) \in \mathbb{R}^3$, regressed in log-space to improve numerical stability across the wide range of object sizes encountered in traffic scenes (from bicycles to buses);
- **Orientation** $(\sin\alpha, \cos\alpha) \in \mathbb{R}^2$: encoding the yaw angle as its sine and cosine avoids the π-periodicity discontinuity of direct angle regression, making the learning target smooth and continuous;
- **Velocity** $(v_x, v_y) \in \mathbb{R}^2$: estimated by the velocity head (see Section 2.5) for temporal association in tracking.

All regression heads are supervised with an L1 loss applied exclusively at ground-truth center locations. This is a notable design choice: rather than densely supervising all spatial locations as in anchor-based methods, only the positive locations contribute to the regression loss, which keeps the training signal focused and well-defined.

After decoding all box parameters from the regression outputs at each peak location, Non-Maximum Suppression is applied to remove duplicate detections.

Figure 3 illustrates the predicted heatmaps for six object categories, highlighting class-specific confidence patterns.

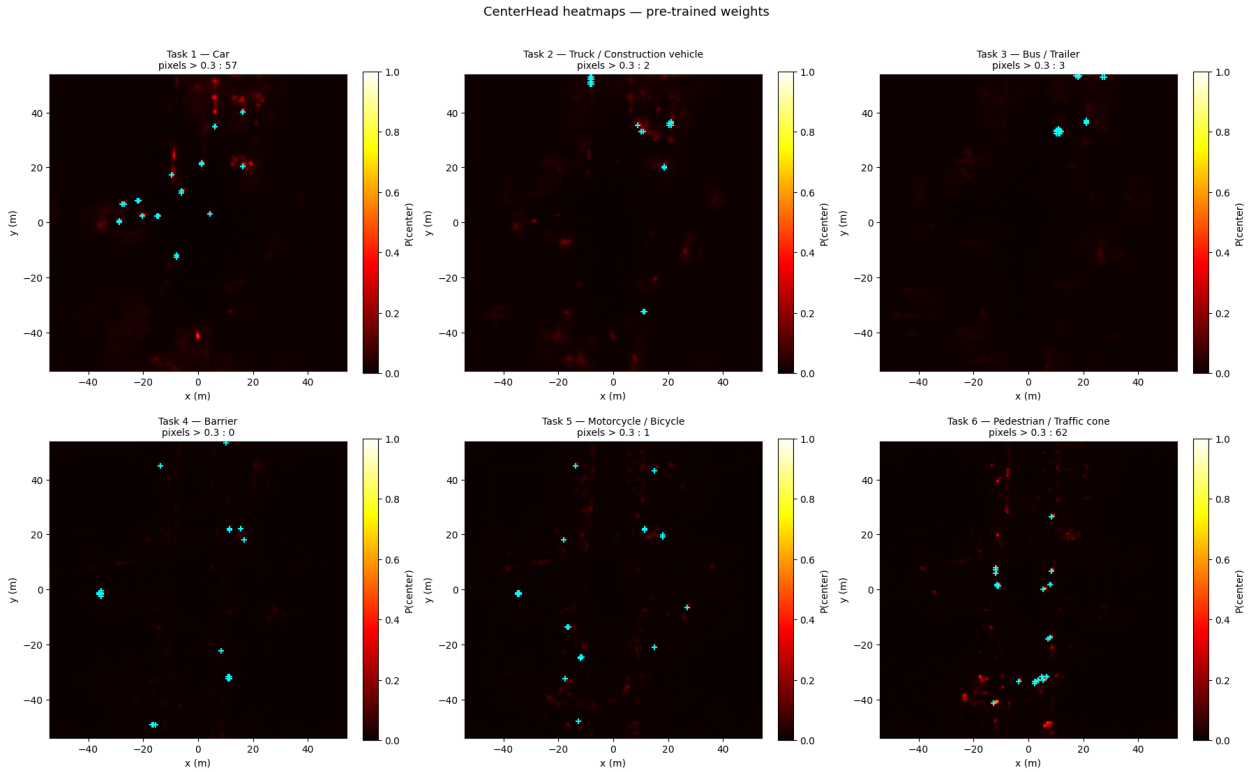

Figure 3: Predicted CenterHead heatmaps for six object categories on a nuScenes validation frame. Bright peaks correspond to high-confidence detected centers; cyan crosses indicate ground-truth object centers (all classes). Car and pedestrian/traffic cone categories (Tasks 1 and 6) dominate with 57 and 62 detections respectively, while heavier vehicles (Tasks 2–3) produce only sparse, sharply localized peaks. Barrier (Task 4) yields no detection, as this class is absent from the two mini-val scenes.

2.4 Two-Stage Refinement

Although the first-stage head predicts complete 3D boxes, all properties are inferred from a single center feature. This is a fundamental limitation: in practice, the LiDAR sensor typically observes the side of an object rather than its center, so the center feature alone may lack sufficient geometric information for precise localization. CenterPoint addresses this with a lightweight second refinement stage.

Surface-center feature extraction. For each first-stage proposal, 5 points are defined: the predicted box center plus the centers of its four outward-facing faces (top and bottom faces are excluded as they project to the same BEV location as the box center). For each point, a feature vector is extracted via bilinear interpolation from M, then the 5 vectors are concatenated and passed through a shared two-layer MLP producing two outputs:

- **A class-agnostic confidence score** $\hat{I}_t$, trained with binary cross-entropy against an IoU-guided target:
$$I_t = \min\big(1,\ \max(0,\ 2 \cdot \mathrm{IoU}_t - 0.5)\big) \tag{3}$$
- **A box regression refinement**, trained with L1 loss on top of first-stage predictions.

At inference, the final confidence is the geometric mean of both stages:

$$\hat{Q}_t = \sqrt{\hat{Y}_t \cdot \hat{I}_t} \tag{4}$$

Efficiency. By operating on the 2D BEV feature map rather than on 3D voxel features, the second stage adds only 6–7 ms per frame. The authors report in their ablation study (Table 9 of the original paper) a +2 mAPH gain on Waymo (64-beam LiDAR, ∼180k points/frame), but no improvement on nuScenes (32-beam, ∼30k points/frame). They attribute this to the sparser point cloud on nuScenes, which provides fewer surface cues for the refinement module to exploit.

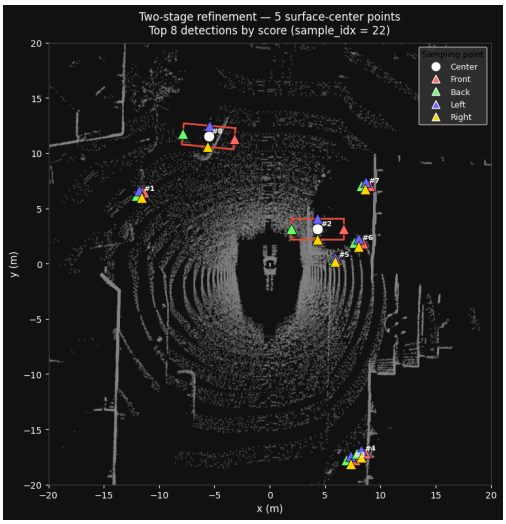

Figure 4: The 5 surface-center sampling points for the top-8 detections on a nuScenes frame. Box center (white circle), front/back/left/right face centers (colored triangles, see legend). For large objects, the face-center points probe distinctly different surface regions; for small objects, the 5 points nearly coincide.

2.5 Velocity Head and Greedy Tracking

A LiDAR-based detector like CenterPoint operates frame by frame, producing independent detections with no memory of previous frames. The naive tracking approach, matching detections by

proximity, fails in practice: at 2 Hz the nuScenes sensor leaves 0.5 s between annotated frames, during which a pedestrian can move 1 m and a car up to 7 m. Without knowing *where an object is heading*, position-only matching produces excessive false associations.

Velocity head. CenterPoint addresses this by adding a regression head that predicts, for each detected object, its 2D ground-plane velocity $\mathbf{v} = (v_x, v_y) \in \mathbb{R}^2$ in m/s. Unlike other regression branches, velocity is an inter-frame quantity that cannot be inferred from a single snapshot: the head therefore requires two consecutive BEV feature maps as input so the network can learn motion from the change in scene appearance. On nuScenes, this temporal signal is provided implicitly through 10-sweep aggregation, where each LiDAR point carries a timestamp offset as a 5th feature. The velocity target is the inter-frame displacement divided by Δt, supervised with an L1 loss at ground-truth center locations.

Greedy tracking. At each new frame t, every detection i is projected backward to $t-1$:

$$\hat{p}_i(t \to t-1) \;=\; \hat{p}_i(t) - \hat{v}_i \cdot \Delta t \tag{5}$$

and matched to the nearest active track by Euclidean distance, subject to a class-specific threshold τ_c (1 m for pedestrians, up to 13 m for motorcycles) and same-class constraint. Unmatched detections spawn new tracks; unmatched tracks are kept alive for up to $T = 3$ frames with their center propagated by last known velocity, then deleted.

Impact. The authors report (Table 12 of the original paper) that this design runs in ∼1 ms per frame and outperforms a Mahalanobis-distance Kalman filter tracker by +3.7 AMOTA on nuScenes validation when given identical detections, demonstrating that a learned velocity estimate provides more reliable motion cues than a kinematic state estimated online from noisy positions.

3 Experiments

3.1 Setup and Evaluation Metrics

We evaluate CenterPoint (VoxelNet) on the nuScenes mini validation split using the authors' pre-trained `centerpoint_voxel_1440` checkpoint. Full training was not feasible given the dataset scale (∼300 GB) and compute requirements; we therefore restrict experiments to inference on 2 scenes. The configuration is summarized in Table 1.

Model	CenterPoint (VoxelNet)
Dataset	nuScenes mini validation (2 scenes)
Voxel size	$0.075 \times 0.075 \times 0.2$ m
BEV grid	1440×1440
LiDAR sweeps	10 aggregated

Table 1: Experimental configuration.

The nuScenes benchmark uses two main evaluation metrics. The **mean Average Precision (mAP)** is computed using BEV center-distance thresholds $d \in \{0.5, 1, 2, 4\}$ m rather than IoU,

which better reflects localization quality for rotated 3D boxes. The **nuScenes Detection Score (NDS)** combines mAP with five True Positive error metrics:

$$\text{NDS} = \frac{1}{10}\Big[5 \cdot \text{mAP} + \sum_{e \in E}(1 - \min(1, e))\Big] \tag{6}$$

where $E = \{\text{ATE}, \text{ASE}, \text{AOE}, \text{AVE}, \text{AAE}\}$ denote translation, scale, orientation, velocity, and attribute errors on true positive detections respectively.

3.2 Results from the Original Paper

On the nuScenes full validation set (150 scenes), the `centerpoint_voxel_1440` checkpoint reports 56.4 mAP and 64.8 NDS (Table 6 of the original paper). On the Waymo Open Dataset, CenterPoint achieves 71.8 level-2 mAPH for vehicles and 66.4 for pedestrians, surpassing all prior LiDAR-only submissions by +7.1 and +10.6 mAPH respectively.

The paper's ablation studies further show that simply switching from an anchor-based to a center-based head, with identical backbones and training, yields a consistent +4.3–4.5 mAPH improvement on Waymo and +3.8–4.1 mAP on nuScenes across both VoxelNet and PointPillars encoders, confirming that the performance gain is primarily attributable to the representation change rather than other architectural choices.

3.3 Our Inference Results

3.3.1 Detection Performance

Table 2 compares our inference results on the nuScenes mini validation split with the official pretrained checkpoint.

Model	mAP (%)	NDS (%)
CenterPoint (`centerpoint_voxel_1440`, full val – 150 scenes)	56.4	64.8
CenterPoint (ours, mini val – 2 scenes)	55.4	57.2

Table 2: Comparison with the official pretrained checkpoint on the full validation set (Table 6 of the original paper).

The mAP gap of ∼1 point is negligible and confirms that the pretrained checkpoint is correctly loaded and evaluated on our subset.

The NDS gap of 7.6 points is considerably larger and requires a more careful explanation. NDS aggregates mAP with five True Positive error metrics, including velocity error (AVE). On a 2-scene mini split, rare classes such as trailer, barrier, and construction vehicle are entirely absent, causing their TP errors to default to 1.0 and heavily penalizing NDS. This gap does not reflect a failure of the model or the evaluation pipeline, but is a direct consequence of the mini split size: with only 2 scenes, a single absent class is sufficient to significantly drag down the NDS. The mAP gap of ∼1 point, which is insensitive to this effect, confirms that the checkpoint is correctly loaded and evaluated.

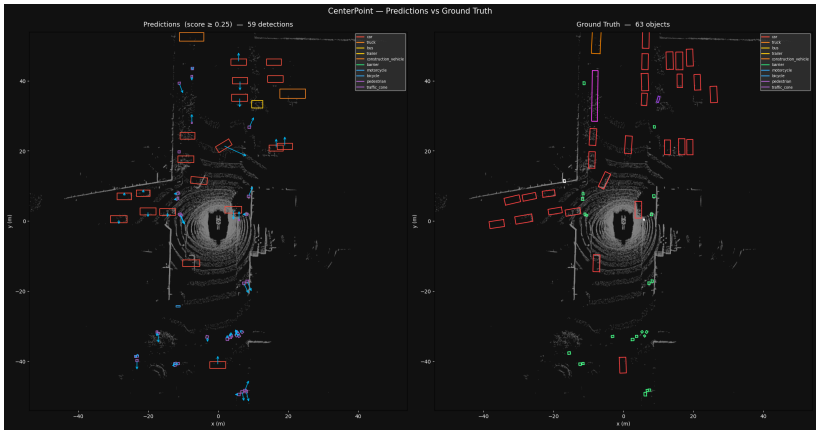

Figure 5: BEV detection results on a nuScenes mini validation frame. Left: CenterPoint predictions (score ≥ 0.25, 59 detections). Right: ground truth (63 objects).

3.3.2 Per-Class Analysis

Per-class Average Precision at all four distance thresholds is shown in Figure 6. Well-represented classes (car, bus, pedestrian, truck) achieve high AP. Several categories (trailer, construction vehicle, barrier) obtain zero AP because they are entirely absent from the two mini-val scenes — a direct consequence of nuScenes' long-tail distribution, not a model failure.

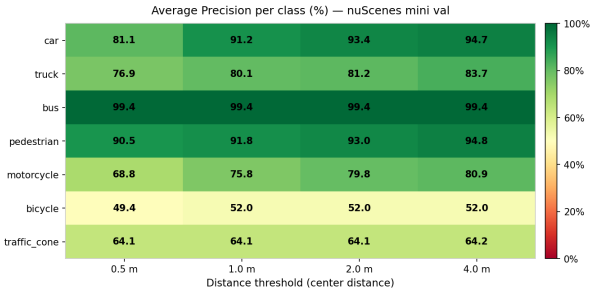

Figure 6: Average Precision per class on the nuScenes mini validation set at four center-distance thresholds.

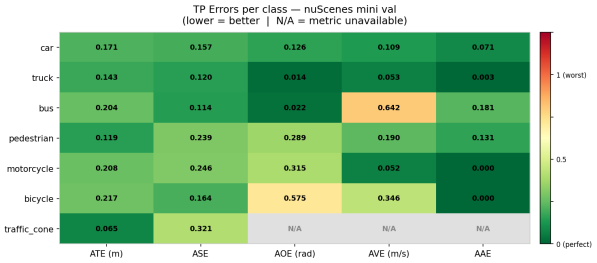

Figure 7: True Positive localization errors per class. Lower is better. ATE: translation error (m); ASE: scale error; AOE: orientation error (rad); AVE: velocity error (m/s); AAE: attribute error.

Bicycle exhibits a notably high AOE (0.575 rad): bicycles are narrow objects that can be observed from any direction, making yaw estimation geometrically harder from sparse LiDAR returns. **Bus** shows a high AVE (0.642 m/s), likely because buses in this split are stationary at stops while the model, trained predominantly on moving buses, predicts non-zero velocity. **Motorcycle** reports AVE = 0.000, an artifact of the tiny sample size rather than a genuine perfect prediction.

4 Critical Discussion

4.1 Strengths

The central contribution of CenterPoint is to demonstrate that the performance gap between anchor-based and center-based detectors is primarily attributable to the representation itself rather than to architectural complexity. The ablation studies in Tables 5 and 6 of the original paper confirm a consistent improvement of 4.3–4.5 mAPH on Waymo and 3.8–4.1 mAP on nuScenes across both VoxelNet and PointPillars encoders, with identical backbones and training schedules. What makes this result credible is that the gains are not uniformly distributed: they concentrate precisely on the categories where axis-aligned anchors fail most severely. On Waymo, pedestrian detection at 30–45° heading angles improves by 9.2 mAPH (Table 7); on nuScenes, the largest gains appear on bicycles (+6.4 mAP), construction vehicles (+7.0 mAP), and traffic cones (+5.6 mAP), objects with atypical aspect ratios or arbitrary orientations. This asymmetry is not coincidental: it is exactly what the center-based formulation predicts, and it constitutes genuine evidence for the claim.

A less obvious strength is the PKL score on the nuScenes test set, where CenterPoint achieves 0.69 against 0.77 for the next best published method. The PKL metric measures how much a set of detections perturbs a downstream motion planner's trajectory relative to ground truth, it was not optimized during training and was only revealed after leaderboard submission. Outperforming on a metric that was explicitly hidden from the training objective suggests that center-based detections produce fewer safety-critical false positives, which is arguably the property that matters most in an autonomous driving context.

4.2 Limitations

The paper claims that center-based representation allows the backbone to learn rotational invariance "more naturally". This phrasing deserves scrutiny. The backbone operates on a fixed-orientation BEV grid anchored to the ego-vehicle coordinate frame: there is no architectural invariance by construction, only a reduction of the assignment bias present in anchor-based training. Table 7 confirms this directly, vehicle detection degrades from 67.8 to 51.6 mAPH when heading angle shifts from 0–15° to 30–45°, a relative drop of over 23%. What CenterPoint achieves is a softer training objective that does not penalize rotated objects as harshly during target assignment, which is a real and measurable improvement, but it falls well short of the invariance the paper implicitly suggests. True rotational equivariance would require a backbone designed for it, which CenterPoint does not employ.

A related methodological concern is that the anchor-based versus center-based ablation (Tables 5 and 6) is not as clean as it appears. Switching from anchors to centers does not only change the output representation: it simultaneously changes the training objective, replacing IoU-based target assignment with class-specific thresholds by a Gaussian heatmap loss with per-location supervision. These are fundamentally different training signals, and attributing the entire performance gain to the representational change alone overstates what the ablation actually demonstrates. It is plausible that part of the improvement comes from the denser and more stable supervision provided by the heatmap formulation rather than from the center-based representation per se.

The evaluation protocol on nuScenes introduces a further confound. The benchmark measures mAP using BEV center-to-center distance thresholds at 0.5, 1, 2, and 4 meters rather than 3D IoU. This choice is not neutral: center-based detection is by design optimized for center localization precision, which is exactly what the metric rewards. An anchor-based detector with more accurate box shape estimation but marginally noisier center predictions would appear disproportionately worse under this protocol. The paper does not discuss the implications of this metric selection, and the comparisons in Tables 5 and 6 should be interpreted with this in mind.

The two-stage refinement is presented as a general contribution, but the ablation reveals a more conditional picture. On Waymo, with a 64-beam LiDAR producing approximately 180k points per frame, the surface-center features yield roughly 2 mAPH gain at minimal computational cost. On nuScenes, with a 32-beam sensor and approximately 30k points per frame, the improvement is negligible. This gap is not incidental: the second stage can only extract meaningful surface cues when point density is sufficient to resolve object geometry around the proposal. For small objects processed via PointPillars, a pedestrian may occupy a single voxel column in the BEV grid, causing all five surface-center sampling points to collapse to essentially the same location. Presented as a broadly applicable efficiency gain, the second stage is in practice a sensor-dependent component whose contribution must be re-evaluated in any new deployment context, particularly as LiDAR hardware continues to vary significantly in beam count and point density.

The tracking module raises two concerns that the paper does not fully address. The reported +3.7 AMOTA improvement over the Kalman-filter tracker of Chiu et al. [1] is partially confounded: better detections fed into any tracker will improve tracking metrics, making it difficult to isolate the contribution of the matching algorithm itself. Beyond this, the greedy assignment strategy, processing detections in decreasing confidence order and making irrevocable sequential decisions, is a heuristic. The Hungarian algorithm would yield a globally optimal assignment under any fixed cost function at comparable computational cost, and the choice of greedy matching is justified only by its simplicity. In dense scenarios such as crowded pedestrian areas or parking lots, where multiple nearby tracks compete for the same detections, sequential assignment predictably produces more identity switches than global optimization would.

Finally, every newly appearing object enters the pipeline with no prior motion history, making its first velocity estimate unreliable by construction. For that initial frame, the backward projection used for matching reduces to simple proximity-based association, precisely the naive approach the paper argues is insufficient at nuScenes' 0.5-second inter-frame interval. The paper reports 760 identity switches on the test set but provides no breakdown distinguishing cold-start failures from tracking failures under occlusion or trajectory crossing. This distinction would be informative and is left unaddressed.

References

[1] Hsu-kuang Chiu, Antonio Prioletti, Jie Li, and Jeannette Bohg. Probabilistic 3d multi-object tracking for autonomous driving. *arXiv preprint arXiv:2001.05673*, 2020.

[2] Hei Law and Jia Deng. Cornernet: Detecting objects as paired keypoints. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018.

[3] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense

object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.

[4] Yan Yan, Yuxing Mao, and Bo Li. Second: Sparsely embedded convolutional detection. In *Sensors*, 2018.

[5] Tianwei Yin, Xingyi Zhou, and Philipp Krähenbühl. Center-based 3d object detection and tracking. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[6] Xingyi Zhou, Dequan Wang, and Philipp Krähenbühl. Objects as points. *arXiv preprint arXiv:1904.07850*, 2019.